

HW1 - Seeded Circuits, Bit Flips, and Measurement Mapping

Context: AI-resilient, auto-gradable Qiskit assignment prototype for quantum computer programming.

Deliverables: completed notebook, answers.json, and optional circuit.qpy if requested by the instructor.

Recommended environment: Google Colab with qiskit and qiskit-aer installed.

1. Purpose

This assignment introduces the shared structure used throughout the assignment set: deterministic seeds, personalized circuit artifacts, machine-readable answers.json output, and hidden reference grading. Your task is to build a small Qiskit circuit whose output depends on your personalized seed.

The assignment is designed to expose common mistakes that text-only LLMs and students may make, especially bit-ordering confusion and incorrect measurement mapping.

2. Learning outcomes

- Generate a personalized circuit configuration from a student-specific seed.
- Prepare a computational basis state using X gates.
- Apply a deterministic bit-flip mask.
- Measure qubits into classical bits using a specified measurement map.
- Explain why Qiskit count strings are displayed as `c[n-1]...c[0]`.
- Export answers.json for an autograder.

3. Assignment summary

Your seed generates a configuration containing:

- `num_qubits`: either 4 or 5;
- `initial_bits_q0_to_qn`: the initial bit pattern in `q[0]`, `q[1]`, ... order;
- `flip_mask_q0_to_qn`: which qubits must be flipped with X gates;
- `final_bits_q0_to_qn`: the expected qubit values after bit flips;
- `measurement_map`: pairs of `[qubit_index, classical_bit_index]`;
- `measurement_mode`: direct or reversed.

4. Required circuit

Your circuit must follow these steps exactly:

- Create `n` qubits and `n` classical bits.
- For each 1 in `initial_bits_q0_to_qn`, apply X to the corresponding qubit.
- Add a barrier labelled seeded initial state.
- For each 1 in `flip_mask_q0_to_qn`, apply X to the corresponding qubit.
- Add a barrier labelled seeded bit flips.
- Measure according to `measurement_map`. Do not use `measure_all()` for this assignment.

5. Bit-order warning

Qiskit count strings are displayed in classical-bit order $c[n-1] \dots c[0]$. This is not necessarily the same as $q[0] \dots q[n-1]$. If your measurement map is reversed, the displayed histogram key may look reversed relative to the qubit index order.

Your reflection must explicitly explain how each measured qubit maps to a classical bit and why the displayed bitstring is correct.

6. answers.json requirements

Your final notebook must export answers.json with at least these fields:

Field	Meaning
assignment_id	Should be HW1
student_id	Your identifier used to generate the seed
seed	Deterministic seed from student_id
num_qubits	Number of qubits in your circuit
measurement_map	List of [qubit_index, classical_bit_index] pairs
initial_bits_q0_to_qn	Initial bit pattern in $q[0] \dots q[n-1]$ order
flip_mask_q0_to_qn	Seeded X-flip mask
final_bits_q0_to_qn	Initial bits XOR flip mask
expected_display_bitstring	Expected histogram key in $c[n-1] \dots c[0]$ order
counts	Simulator counts dictionary
dominant_bitstring	Most frequent simulator output
reflection_bit_order	Explanation of bit ordering
reflection_bit_flip	Explanation of the bit-flip mask

7. Grading checklist

- Correct personalized seed and configuration used.
- Circuit implements initial X gates and seeded bit flips.
- Measurement map is implemented correctly.
- Simulator dominant bitstring equals expected_display_bitstring.
- answers.json validates against schema_hw1.json.
- Reflection explains bit ordering and bit flips clearly.

8. Optional extension

The same circuit may be run on IBM Quantum hardware as a later extension. If hardware is used, students should report backend name, job ID, raw counts, dominant bitstring, and whether hardware noise affected the expected output. HW1 itself is designed to be reliably autograded using simulation.